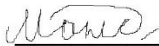


Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
ТОМСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ СИСТЕМ
УПРАВЛЕНИЯ И РАДИОЭЛЕКТРОНИКИ (ТУСУР)
Кафедра экономической математики, информатики и статистики (ЭМИС)

ТИПЫ ПРЕДИКАТОВ. ТИПОВЫЕ ЗАДАЧИ
Отчет по практической работе по дисциплине «Основы интеллектуальных
систем»

Студент гр. 591-1

 Д.Ю. Мандриков

«7» мая 2024 г.

Руководитель

К.т.н., доцент каф ЭМИС

_____ Н.Ю. Истомина

оценка

«__» _____ 2024 г.

Томск 2024

Практическая работа №4

«Типы предикатов. Типовые задачи»

Цель работы: знакомство с синтаксисом и получение практических навыков составления простейших программ на языке логического программирования – ПРОЛОГ.

Краткие теоретические сведения.

Переменные и константы. Исходными элементами описания предметной области в приведенных выше примерах являются отдельные утверждения: простые – факты и сложные – правила. При этом не рассматривались состав и структура утверждения в смысле связи действия, агента, объекта и других грамматических составляющих предложения. Однако не всегда возможно адекватно описать предметную область таким простым способом. Для углубленного анализа и описания предметной области язык программирования должен иметь возможность работы с константами, переменными и сложными структурами. ПРОЛОГ обладает такими возможностями.

Большинство ПРОЛОГ-трансляторов работают только с латинским алфавитом в качестве имен предикатов, поэтому для записи имен предикатов используется латинская нотация. Совокупность фактов называется базой данных. Этот термин будет использоваться для объединения фактов при описании предметной области для решения конкретной задачи.

Во многих версиях ПРОЛОГа приняты следующие соглашения: заглавные буквы обозначают переменные, а строчные буквы, целые или действительные числа, или любые символы, заключенные в кавычки, обозначают постоянные значения – константы.

Отличия между Visual Prolog и традиционным Прологом. Различия между традиционным Прологом и Visual Prolog можно провести по следующим категориям: Различия в структуре программы: Различия между Visual Prolog и традиционным Прологом имеются, но они не существенны.

Они сводятся к пониманию того, как различаются декларации (declarations) от определений (definitions), а также к явному выделению цели Goal. Все это поддержано специальными ключевыми словами.

Файловая структура программы: Visual Prolog предоставляет возможности структуризации программ с использованием файлов различного типа.

Границы видимости: программа системы Visual Prolog может поддерживать функционирование, располагающееся в различных модулях с использованием концепции идентификация пространств.

Объектная ориентированность: программа на языке Visual Prolog может быть написана как объектно-ориентированная программа с использованием классических свойств объектно-ориентированной парадигмы.

В Прологе, когда необходимо использовать предикат, то это делается без каких-либо предварительных указаний движку Пролога о таких намерениях. Например, в предыдущих руководствах клауз предиката grandFather (дедушка) был непосредственно записан с использованием традиционной для Пролога конструкции голова-тело.

Аналогично, когда в традиционном Прологе требуется использовать составной домен, можно его использовать без предупреждения движка по поводу этого намерения. Просто используется домен тогда, когда в этом возникает необходимость.

Однако, в Visual Prolog, перед написанием кода для тела клаузы предиката необходимо сначала объявить о существовании такого предиката компилятору. Аналогично, перед использованием любых доменов они должны быть объявлены и представлены компилятору.

Причиной такой необходимости в предупреждениях является попытка как можно раньше обнаружить возможность исключений периода исполнения.

Под "исключениями периода исполнения (runtime exceptions), понимаются события, возникающие только во время исполнения программы. Например, если использовать целое число в качестве аргумента функтора, а

вместо этого по ошибке использовали вещественное число, то в процессе исполнения возникла бы ошибка периода исполнения (в программах, написанных для ряда компиляторов, но не для Visual Prolog) и программа в этом случае завершилась бы неуспешно.

Когда объявляются предикаты или домены, которые определены, то появляется своего рода позиционная грамматика (какому домену принадлежит какой аргумент), доступная компилятору. Более того, когда Visual Prolog выполняет компиляцию, он тщательно проверяет программу на наличие таких грамматических ошибок, наряду с другими.

Благодаря этому свойству Visual Prolog, повышается конечная эффективность программиста. Программист не должен ждать, когда реально работающая программа совершит ошибку. Часто конкретная последовательность событий, приводящая к исключению периода исполнения кажется настолько неуловимой, что ошибка может проявиться через много лет, или она может заявить о себе в критической ситуации и привести к непредсказуемым последствиям!

Все это автоматически ведет к тому, что компилятор должен получать точные инструкции по поводу предикатов и доменов в виде соответствующих объявлений, которые должны предшествовать определениям.

Программа на Visual Prolog, представляемая кодом, разделяется ключевыми словами на секции разного вида путем использования ключевых слов, предписывающих компилятору, какой код генерировать. К примеру, есть ключевые слова, обозначающие различие между декларациями и определениями предикатов и доменов. Обычно, каждой секции предшествует ключевое слово. Ключевых слов, обозначающих окончание секции, нет. Наличие другого ключевого слова обозначает окончание предыдущей секции и начало другой.

Исключением из этого правила являются ключевые слова `implement` и `end implement`. Код, содержащийся между этими ключевыми словами, есть код, который относится к конкретному классу.

В этом руководстве используются следующие ключевые слова: `implement` и `end implement`

Среди всех ключевых слов, обсуждаемых здесь, это единственные ключевые слова, используемые парно. Visual Prolog рассматривает код, помещенный между этими ключевыми словами, как код, принадлежащий одному классу. За ключевым словом `implement` обязательно ДОЛЖНО следовать имя класса.

`Orep` – это ключевое слово используется для расширения области видимости класса. Оно должно быть помещено вначале кода класса, сразу после ключевого слова `implement` (с именем класса и, возможно именем интерфейса - прим. перев.).

`Constants` – это ключевое слово используется для обозначения секции кода, которая определяет неоднократно используемые значения, применяемые в коде. Например, если строковый литерал "PDC Prolog" предполагается использовать в различных местах кода, тогда можно единожды определить мнемоническое (краткое, легко запоминаемое слово) для использования в таких местах.

`Domains` – Это ключевое слово используется для обозначения секции, объявляющей домены, которые будут использованы в коде. Синтаксис таких объявлений позволяет порождать множество вариантов объявлений доменов, используемых в тексте программы.

Вообще говоря, объявляется функтор, который будет использоваться в качестве домена и ряд доменов, которые используются в качестве его аргументов. Функторы и составные домены рассматриваются в соответствующем руководстве.

`class facts` – это ключевое слово представляет секцию, в которой объявляются факты, которые будут в дальнейшем использоваться в тексте программы. Каждый факт объявляется как имя, используемое для обозначения факта, и набор аргументов, каждый из которых должен соответствовать либо стандартному (предопределенному), либо объявленному домену.

`class predicates` – эта секция содержит объявления предикатов, которые определяются в тексте программы в разделе `clauses`. И опять, объявление предиката – это имя, которое присваивается предикату, и набор аргументов, каждый из которых должен соответствовать либо стандартному (предопределенному), либо объявленному домену.

`Clauses` – среди всех разделов, существующих в тексте программ на Visual Prolog, это единственный раздел, который близко совпадает с традиционными программами на Прологе. Он содержит конкретные определения объявленных в разделе `class predicates` предикатов, причем синтаксически им соответствующие.

`Goal` – этот раздел определяет главную точку входа в программу на языке системы Visual Prolog. Более детальное описание приводится ниже.

`Goal` (цель) – в традиционном Прологе, как только какой-либо предикат определен в тексте, Пролог-машине может быть предписано выполнение программы, начиная с этого предиката. В Visual Prolog это не так. Будучи компилятором, он отвечает за генерацию эффективного исполняемого кода написанной программы. Этот код выполняется не в то же время, когда компилятор порождает код. Поэтому компилятору надо знать заранее точно, с какого предиката начнется исполнение программы. Благодаря этому, когда программа вызывается на исполнение, она начинает работу с правильной начальной точки. Как можно уже догадаться, откомпилированная программа может работать независимо от компилятора Visual Prolog и без участия среды IDE.

Для того, чтобы это стало возможным, введен специальный раздел, обозначенный ключевым словом `Goal`. Его можно представлять, как особый предикат, не имеющий аргументов. Это предикат – именно тот предикат, с которого вся программа начинает исполняться.

Необходимость помещения всех частей большой программы в один файл несомненно является неудобством. Это ведет к тому, что программа становится нечитаемой и иногда неправильной. Visual Prolog предусматривает

возможность деления текста программы на отдельные файлы, используя среду IDE (Integrated Development Environment) и, кроме того, IDE позволяет помещать логически связанные фрагменты текста программы в отдельные файлы. Несмотря на такое разделение программы на части, сущности, находящиеся в общем использовании, доступны.

Например, если имеется домен, который используется несколькими файлами, то объявление домена делается в отдельном файле и к этому файлу из других файлов существует доступ.

Для упрощения этого руководства мы будем использовать один файл для разработки текста программы. На самом деле в процессе построения программы среда IDE создавала бы автоматически несколько файлов, но мы сейчас будем это умалчивать. Об этом написано в других руководствах.

В Visual Prolog текст программы разделен на отдельные части, каждая часть определяется как класс (class). В объектно-ориентированных языках программирования, класс - это пакет кода и ассоциированные с ним данные. Это требует длительных объяснений и сделано в других руководствах. Те, кто не знаком с объектно-ориентированным программированием, может представлять себе класс нестрого как синоним понятия модуль. В Visual Prolog каждый класс обычно помещается в отдельный файл.

В процессе исполнения программы, часто бывает так, что программе может потребоваться вызвать предикат, который определен в другом классе (файле). Аналогично, данные (константы) или домены могут быть востребованы в другом файле.

Visual Prolog позволяет делать такие взаимные ссылки на предикаты или данные используя так называемую концепцию области видимости (scope access). Это может стать понятным на примере. Предположим имеется предикат, называемый `pred1` и определенный в классе называемом `class1` (который помещается в другом файле, согласно стратегии среды IDE), и мы хотим вызвать этот предикат в теле клауза некоторого другого предиката

pred2, находящегося в другом файле (скажем, class2). Тогда вот как предикат pred1 должен бы быть вызван в теле клауза предиката pred2.

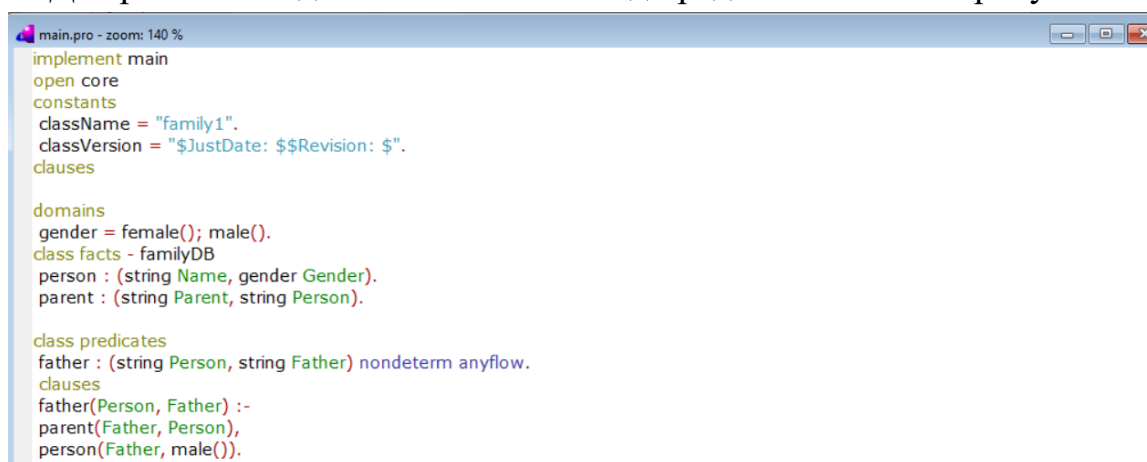
Область видимости класса может быть расширена использованием ключевого слова `open`. Это ключевое слово информирует компилятор о том, что в этот класс должны быть "доставлены" имена (предикатов / констант / доменов), которые были определены в других файлах. Если область видимости расширена, то необходимости использования квалификатора класса (с двойным двоеточием) нет.

Visual Prolog является полностью объектно-ориентированным языком. Весь код, который разрабатывается для программы, помещается в класс. Это происходит само собой, даже если Вы не интересуетесь объектными свойствами языка. Это свойство обнаруживается в примере, разбираемом в этом руководстве. Код помещается в класс, называемый "family1", несмотря на то, что мы не используем объекты, порождаемые этим классом. Кроме того, в этом классе используются общедоступные предикаты, находящиеся в других классах.

Формулировка заданий с полученными результатами.

Задание. На основе представленного примера программы определить ближайших родственников: братьев, сестер, тетей, дядей, племянников, внуков, двоюродных братьев и сестер, внучатых племянников. Исходную базу фактов расширить для получения удачных исходов для всех предикатов.

Для решения задачи был написан код представленный на рисунке 1.



```
main.pro - zoom: 140 %
implement main
open core
constants
  className = "family1".
  classVersion = "$JustDate: $$Revision: $".
clauses

domains
  gender = female(); male().
class facts - familyDB
  person : (string Name, gender Gender).
  parent : (string Parent, string Person).

class predicates
  father : (string Person, string Father) nondeterm anyflow.
clauses
  father(Person, Father) :-
    parent(Father, Person),
    person(Father, male()).
```

Рисунок 1 – Скриншот кода из файла main.pro

Также был составлен файл, описывающий персон и их отношения (рисунок 2).

```
clauses
person("Polina",female()).
person("Anatoliy",male()).
person("Vasiliy",male()).
person("Nadezhda",female()).
person("Elena",female()).
person("Yuri",male()).
person("Lyubov",female()).
person("Denis",male()).
person("Ludmila",female()).
person("Anastasia",female()).
person("Elizabeth",female()).
person("Ekaterina",female()).
person("Anna",female()).
person("Svyatoslav",male()).
person("A",female()).
parent("Polina","Elena").
parent("Anatoliy","Elena").
parent("Vasiliy","Lyubov").
parent("Vasiliy","Yuri").
parent("Nadezhda","Lyubov").
parent("Nadezhda","Yuri").
parent("Elena","Denis").
parent("Elena","Ludmila").
```

Рисунок 2 – Скриншот описания персон и их отношений

Семейное дерево представлено на рисунке 3.

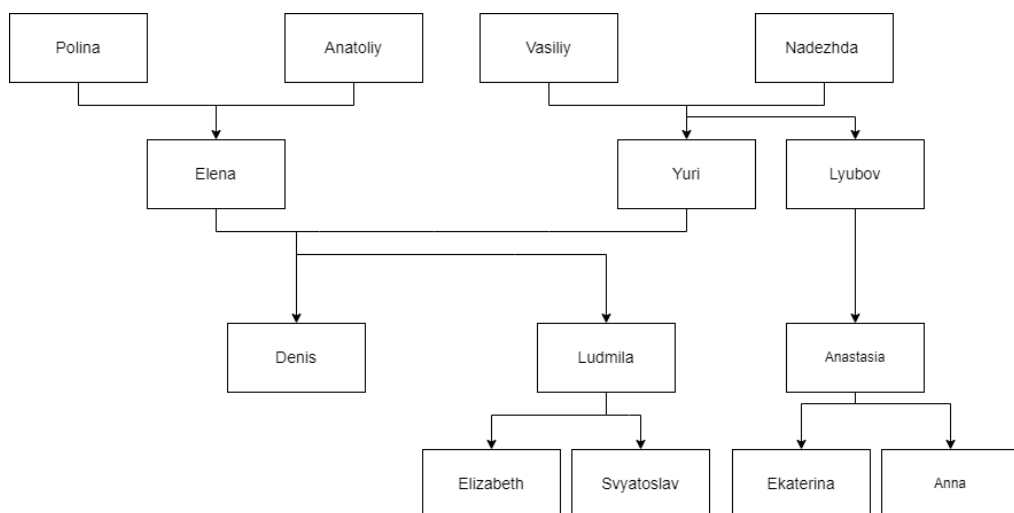
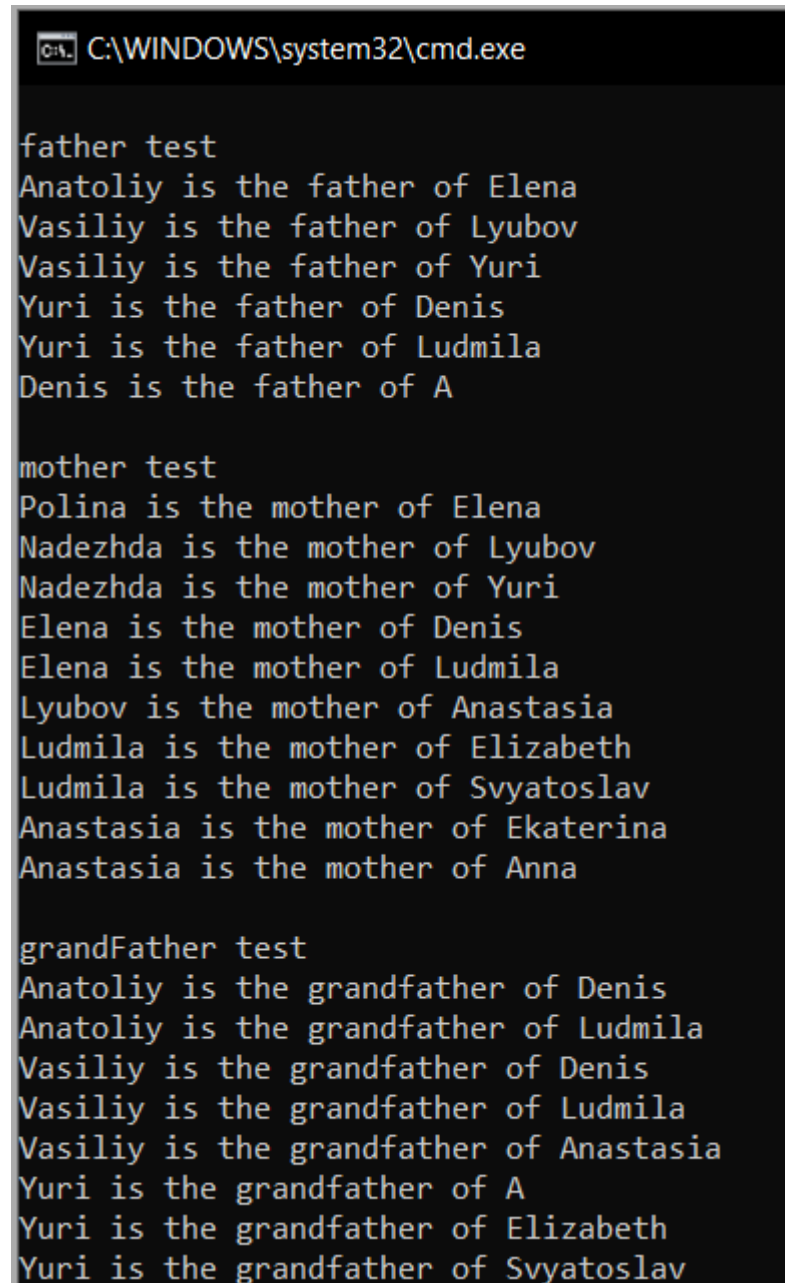


Рисунок 3 – Семейное дерево

Вывод программы представлен на рисунке 4.



```
C:\WINDOWS\system32\cmd.exe

father test
Anatoliy is the father of Elena
Vasiliy is the father of Lyubov
Vasiliy is the father of Yuri
Yuri is the father of Denis
Yuri is the father of Ludmila
Denis is the father of A

mother test
Polina is the mother of Elena
Nadezhda is the mother of Lyubov
Nadezhda is the mother of Yuri
Elena is the mother of Denis
Elena is the mother of Ludmila
Lyubov is the mother of Anastasia
Ludmila is the mother of Elizabeth
Ludmila is the mother of Svyatoslav
Anastasia is the mother of Ekaterina
Anastasia is the mother of Anna

grandFather test
Anatoliy is the grandfather of Denis
Anatoliy is the grandfather of Ludmila
Vasiliy is the grandfather of Denis
Vasiliy is the grandfather of Ludmila
Vasiliy is the grandfather of Anastasia
Yuri is the grandfather of A
Yuri is the grandfather of Elizabeth
Yuri is the grandfather of Svyatoslav
```

Рисунок 4 – Вывод программы

Выводы к лабораторной работе: получены практических навыков составления простейших программ на языке логического программирования – ПРОЛОГ.